

Decoding Strategies

Greedy · Beam · Temperature · Top- k/p · Min- p · Contrastive · Constrained

Outline

The generation problem

Deterministic decoding: greedy & beam

Sampling & temperature

Contrastive search & combining knobs

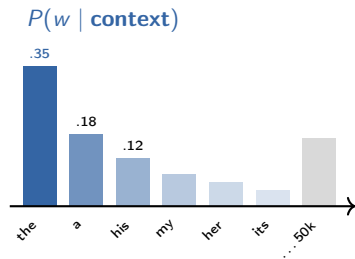
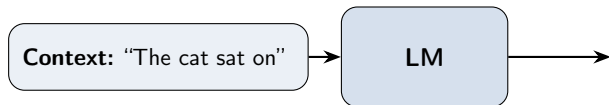
Constrained & speculative decoding

Comparison & practice

The generation problem

The model gives a distribution over the vocabulary. How do we turn it into text?

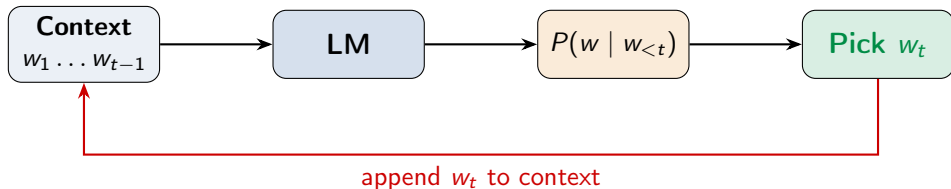
The generation problem



How do we pick the next token?

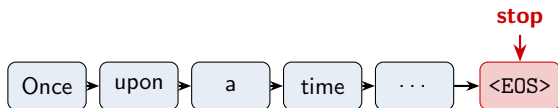
Autoregressive generation

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$



Every decoding method differs **only** in the “Pick w_t ” step.
The model and the loop are always the same.

When does generation stop?



<EOS> token

The model emits the end-of-sequence token — the natural, content-driven stop.

Length cap

`max_new_tokens` / `max_length` reached.
Guards against run-away output.

Time budget

`max_time` exceeded
— finishes the current pass, then halts.

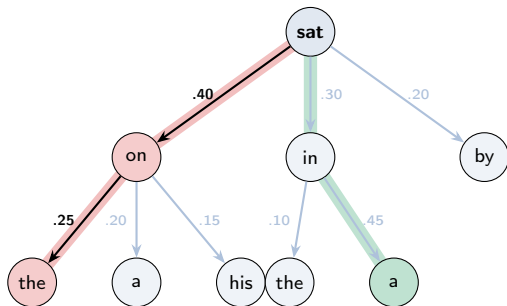
Also `min_new_tokens` (forbid `<EOS>` too early) and custom rules via HuggingFace `StoppingCriteria` — e.g. stop on a delimiter or a given stop-string.

Deterministic decoding: greedy & beam

Always follow the most likely path, and why that is not enough.

Greedy search

$$w_t = \arg \max_w P(w \mid w_1, \dots, w_{t-1})$$



Greedy path
 $.40 \times .25 = .10$

Better path
 $.30 \times .45 = .135$

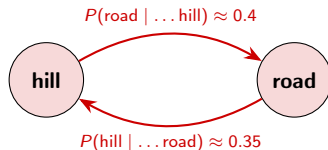
Greedy picks the best token at each step, but can miss the **globally** best sequence.

Only the top few tokens are drawn — the rest of the vocabulary holds the remaining mass, so these do **not** sum to 1.

Greedy search: the repetition trap

Prompt: “Once upon a time, there was a little cat who”

Greedy output: lived in a small house. The house was on the hill. The hill was on the road. The road was on the hill. The hill was on the road. The road was on the hill. The hill was on the ...



Once in a high-probability loop, greedy search **never escapes**.

Each token is locally optimal, but the sequence is degenerate.

Why log-probabilities?

A sequence's probability is a **product** of per-token probabilities — take the log to turn it into a sum:

$$P(w_1, \dots, w_L) = \prod_{t=1}^L P(w_t \mid w_{<t}) \quad \implies \quad \log P = \sum_{t=1}^L \log P(w_t \mid w_{<t})$$

1. No underflow

$0.1^{200} \approx 10^{-200}$ rounds to 0 in floating point. A *sum* of logs stays representable.

2. Products $\rightarrow$ sums

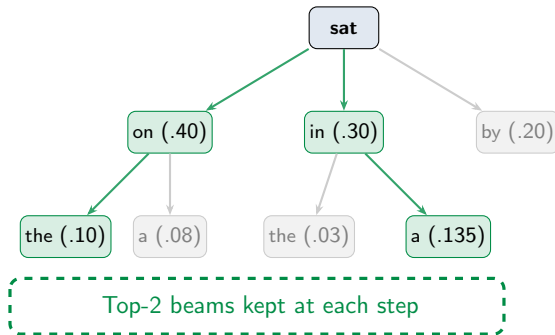
$\log(\prod P) = \sum \log P$.
Adding is cheaper and more stable than multiplying.

3. Same ranking

log is monotonic, so the arg max and the ordering are unchanged.

Since $P \leq 1$, every $\log P \leq 0$ — so scores are **negative**, and “higher” means closer to 0.

Beam search ($B = 2$)



Beam search keeps the B most probable *partial sequences*.

Length-normalized score
(L = sequence length):

$$\text{score} = \frac{1}{L^\alpha} \sum_{t=1}^L \log P(w_t \mid w_{<t})$$

$\alpha \approx 0.6\text{--}0.7$ prevents
preference for short sequences.

Beam search: trade-offs

Works well for:

- Machine translation
- Summarization
- Any task with a “right answer”
- Constrained generation

These tasks want the **most likely** sequence — beam search finds it.

Struggles with:

- Open-ended text / stories
- Dialogue / chat
- Creative writing

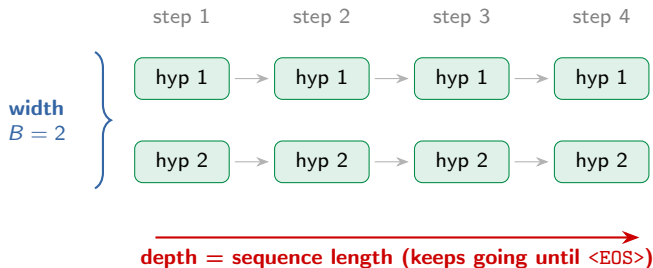
Output is “safe” but **boring**:
generic, repetitive, lacks surprise.

Humans rate beam search text
as less natural than sampling.

Repetition fix: n -gram penalty — if generating token w_t would create an n -gram that already appeared, set $P(w_t) = 0$.

Beam search: width vs depth

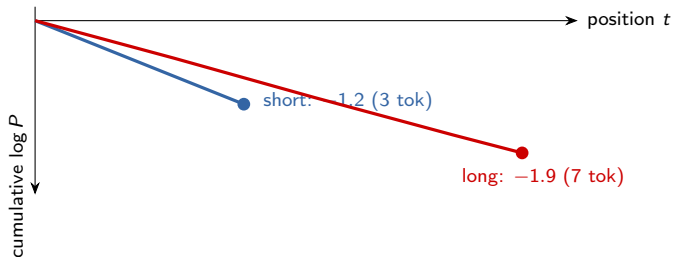
B = how many hypotheses you keep *per step* (width). Depth = how many steps you decode (length). Independent knobs.



$B=3$ keeps **3 rows**, not 3 steps. At the end beam returns the single best full sequence (or the top- n as an “ n -best list”). Chatbots rarely beam-search — they **sample** for diversity; beam suits MT / summarization, where one most-likely answer is wanted.

Beam search: why normalize by length

Each step adds a $\log P \leq 0$, so the running score only **goes down**. A longer sequence scores lower — even when it reads better.



Raw score

$-1.2 > -1.9 \Rightarrow$ **short** wins
just for being short (biased)

Divide by L^α ($\alpha=0.6$)

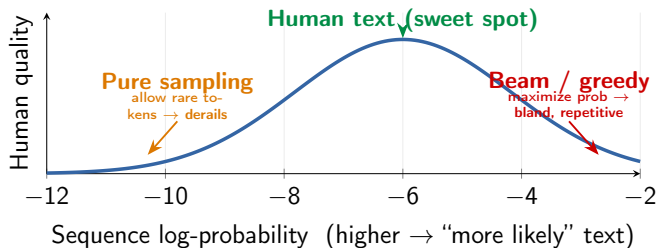
$\frac{-1.2}{3^{0.6}} = -.62$ vs $\frac{-1.9}{7^{0.6}} = -.59$
now the better **long** one wins (fair)

Sampling & temperature

The most probable text is often the dullest. Add controlled randomness.

The probability trap

Why not just maximize probability? Because **text quality vs. sequence probability** is an **inverted U**:



A sequence's log-prob = $\sum_t \log P(w_t \mid \cdot)$; each term is negative (probabilities < 1), so the total is negative — near 0 = high-probability text, more negative = rarer. The -12 to -2 scale is schematic.

Maximizing overshoots into repetition; unrestricted sampling undershoots into nonsense.

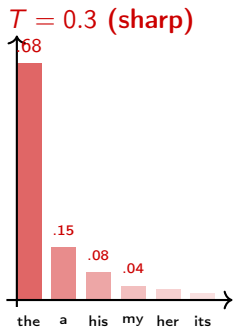
⇒ **good decoding aims for the sweet spot: sample, but cut the unreliable tail** (top- p / min- p).

— Holtzman et al., “The Curious Case of Neural Text Degeneration” (2019)

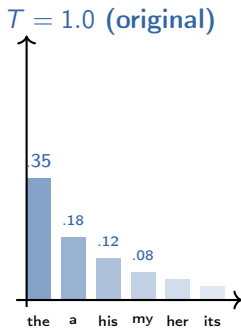
Temperature scaling

$$p_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

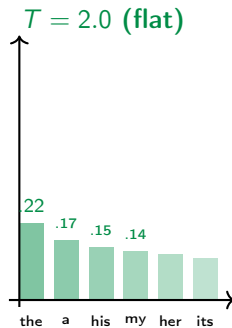
z_i : logits · T : temperature · divide logits by T *before* the softmax



$T \rightarrow 0$: greedy



$T = 1$: unchanged



$T \rightarrow \infty$: uniform

Temperature **reshapes** the distribution — lower = more focused, higher = more random. 17 / 39

Temperature: worked example

Three tokens (*the*, *a*, *his*) with logits $z = (2, 1, 0)$. Plug into $p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$:

$T = 1$ (unchanged):

$$p = \frac{(e^2, e^1, e^0)}{e^2 + e^1 + e^0} = \frac{(7.39, 2.72, 1.00)}{11.11} = (.66, .24, .09)$$

$T = 10$ (hot — shrink the gaps):

$$p = \frac{(e^{0.2}, e^{0.1}, e^0)}{e^{0.2} + e^{0.1} + e^0} = \frac{(1.22, 1.11, 1.00)}{3.33} = (.37, .33, .30) \approx \text{uniform}$$

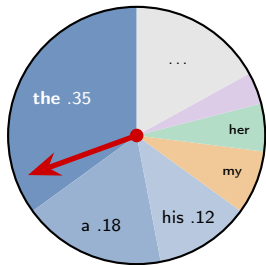
$T \rightarrow 0$ (greedy):

$$\frac{z_i}{T} \rightarrow \infty\text{-sized gaps} \Rightarrow p = (1, 0, 0) \quad (\text{all mass on arg max})$$

Lower T sharpens toward the top token; higher T flattens toward uniform. T is the same softmax — only the logit gaps are rescaled.

Sampling from the distribution

$$w_t \sim P(\cdot \mid w_1, \dots, w_{t-1})$$



Sample = “spin the wheel”

The good: introduces diversity, avoids repetition loops, more human-like

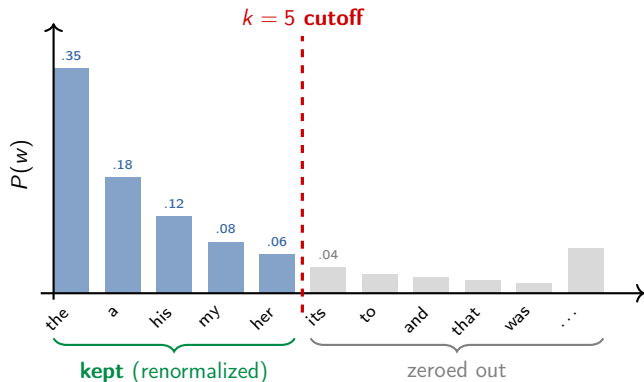
The bad: low-probability tokens (“banana”, “ $\langle \text{unk} \rangle$ ”) occasionally get sampled, **derailing** the entire sequence

“The cat sat on **quantum** and then **refrigerator** began to **oscillate** the ...”

We need to sample, but from a **truncated** distribution.
This motivates top- k and top- p .

Top- k sampling

Idea: Keep only the k most probable tokens, zero out the rest, renormalize.

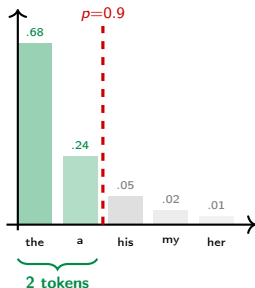


Problem: Fixed k doesn't adapt. When the model is **confident**, $k=50$ includes garbage. When the model is **uncertain**, $k=10$ cuts off good options.

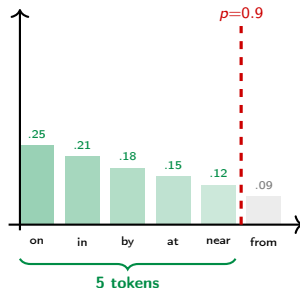
Top- p (nucleus) sampling

Idea: Find the smallest set V_p such that $\sum_{w \in V_p} P(w) \geq p$.

Confident model



Uncertain model

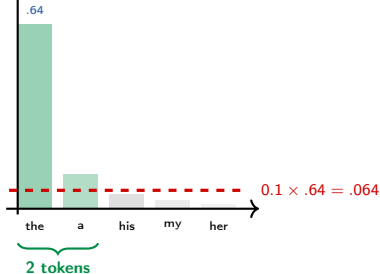


Top- p **adapts** the number of candidates to the model's confidence. This is the key advantage over top- k .

Min- p sampling (2024)

Idea: Set a floor relative to the top token: keep token w iff $P(w) \geq p_{\text{base}} \cdot P(w_{\text{top}})$.

Confident model



Uncertain model



Like top- p , min- p adapts the candidate set to model confidence.
Unlike top- p , the threshold **scales with the top token's probability**:
confident $\rightarrow$ tight filter, uncertain $\rightarrow$ permissive. Single parameter $p_{\text{base}} \in [0, 1]$.

— Nguyen et al., “Turning up the Heat: Min-p Sampling for Creative and Coherent LLM Outputs” (2024)

Contrastive search & combining knobs

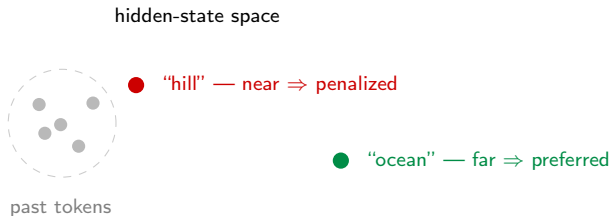
Kill repetition without killing coherence.

Contrastive search: the idea

The dilemma so far:

- ▶ **Greedy / beam** — maximize probability $\Rightarrow$ coherent but **repetitive** (“the hill $\rightarrow$ the road $\rightarrow$ the hill ...”).
- ▶ **Sampling** — inject randomness $\Rightarrow$ diverse but sometimes **incoherent**.

Contrastive search: get *both, deterministically*. At each step prefer a token that is **likely** under the model *but not too similar* to what was already generated.



Contrastive search: the score

$$x_t = \arg \max_{v \in V^{(k)}} \left\{ \underbrace{(1 - \alpha) p_{\theta}(v \mid x_{<t})}_{\text{model confidence}} - \underbrace{\alpha \max_{j < t} s(h_v, h_{x_j})}_{\text{degeneration penalty}} \right\}$$

Model confidence

$p_{\theta}(v \mid x_{<t})$ over the **top- k** candidates $V^{(k)}$.

Keeps the token likely, on-topic, grammatical.

Degeneration penalty

$\max_j s(h_v, h_{x_j})$: largest **cosine similarity** of v 's hidden state to any past token's.

High = echoes the context $\Rightarrow$ punished.

α balances the two terms: $\alpha = 0$ recovers **greedy**; typical $\alpha \in \{0.4, 0.5, 0.6\}$, $k \in \{5, \dots, 15\}$.

Contrastive search: worked example

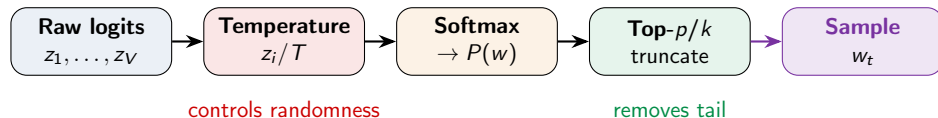
The context already contains “hill”. Score the top- k candidates with $\alpha = 0.6$:

Candidate	confidence p	penalty (sim)	score = $.4 p - .6 \text{ sim}$
“hill” (repeat)	.40	.90	−.38
“road”	.30	.60	−.24
“ocean” (fresh)	.25	.15	+.01 ← chosen

The *most probable* token (“hill”) loses — the penalty kills the repeat; a fresh, still-plausible token wins.

Sibling: **contrastive decoding** — pick tokens maximizing a large “expert” — small “amateur” log-prob gap.

Combining the knobs



Common combinations:

- Temperature T reshapes the distribution *before* truncation
- Top- k and top- p can be used *together* (intersection of both filters)
- Greedy = $T \rightarrow 0$, or equivalently top- k with $k=1$

Constrained & speculative decoding

Force valid output, and make generation faster.

Structured & constrained decoding

Idea: at each step, **mask out** tokens that would break a grammar or schema. The output is *guaranteed* to conform.

JSON mode

Force valid JSON: only allow tokens that keep it well-formed.

- Native in OpenAI, Anthropic, vLLM
- Backbone of tool / function calling

Grammar-guided

Give a CFG or regex; decode only matching tokens.

- **Outlines** (2024): regex/schema
→ mask
- **Guidance** (Microsoft): mix free + constrained

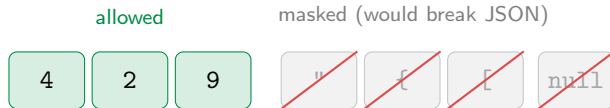
Orthogonal to sampling — stacks with temperature, top- p , min- p ; it only restricts *which* tokens are candidates.

Constrained decoding: example

Emitting JSON for schema `{"city": string, "year": integer}`. We are here:

```
{"city": "Yerevan", "year": | next token?
```

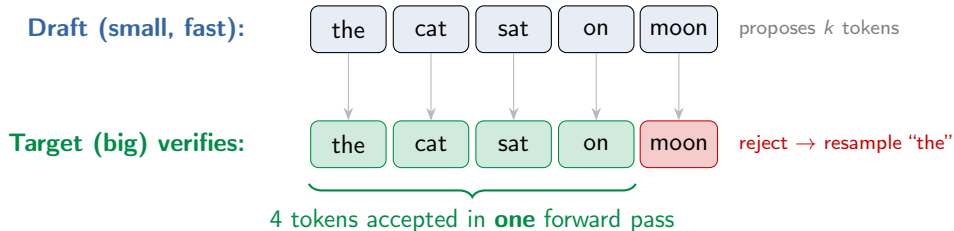
The grammar is now expecting an **integer**, so the decoder masks every token that would break it:



The mask covers the **whole vocabulary** (not just the few shown): every illegal token $\rightarrow -\infty$, then probability is renormalized over the legal ones and the best *legal* token is sampled. A valid grammar always leaves ≥ 1 legal token, so decoding never gets stuck — even if the model “wanted” an illegal token.

Speculative decoding

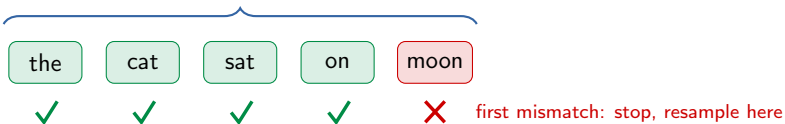
A different axis: not *which* token, but *how fast*. Same output, pure speedup.



Verified in **parallel**: accept the longest matching prefix, resample at the first mismatch. **Lossless** — identical to sampling the target alone, but $2\text{--}3\times$ faster (NVIDIA reports up to $3.6\times$ on H200). *How does it verify k tokens at once, and where does the draft come from? $\rightarrow$ next two slides.*

Speculative decoding: how verification works

target scores all k draft tokens in **one** forward pass



One parallel pass. Causal attention gives the target's next-token distribution at every position at once — the expensive step is paid once, not k times.

Accept / reject, left to right. Greedy target: accept if the draft token = target's arg max. Sampling target: accept x with probability $\min(1, p_{\text{target}}(x)/p_{\text{draft}}(x))$. Stop at the first reject, keep the matching prefix, and let the target fill the corrected token (plus a free bonus token if all k pass).

That accept/reject rule is exactly **rejection sampling** for the target distribution $\Rightarrow$ the output is distributed *identically* to sampling the target alone. That is why it is **lossless**.

Speculative decoding: where the draft comes from

If we only have one model, how do we get a fast “draft”? Four common answers:

Smaller sibling model

A tiny model of the same family drafts (e.g. a 1B drafts for a 70B). The classic setup.

Medusa

Bolt extra lightweight **heads** onto the same model to predict tokens $t+1, t+2, \dots$ at once.

EAGLE / EAGLE-3

Draft at the **hidden-feature** level and extrapolate — higher acceptance; the de facto standard.

Self / n -gram

Early-exit a few of the model's *own* layers, or copy n -grams from the prompt (no extra model).

In production (2026): **EAGLE-3** is the de facto standard — merged into **vLLM**, **SGLang**, and **NVIDIA TensorRT-LLM**, with draft heads trained for **Llama-4**, **Qwen-3**, and **DeepSeek**.
Output unchanged, 2–4× faster.

Comparison & practice

Which knob to turn, and what the API actually exposes.

Comparison of decoding strategies

Method	Deterministic?	Best for	Main weakness
Greedy	Yes	Quick baseline	Repetitive, local optima
Beam search	Yes	Translation, summary	“Boring” open-ended text
Contrastive search	Yes	Coherent open-ended	Extra compute, tune α, k
Temperature	—	<i>Modifier, not standalone</i>	Just a knob
Top- k	No	Open-ended generation	Fixed k doesn't adapt
Top- p	No	Open-ended generation	Slightly slower than top- k
Min- p	No	Open-ended generation	Newer, less battle-tested
Constrained	Either	Structured output (JSON)	Restricts expressiveness

In practice: temperature + top- p is the default (min- p gaining adoption); beam search for translation / summarization; greedy ($T=0$) when deterministic; constrained decoding for JSON / code.

Practical: typical API parameters

```
response = client.generate(  
    prompt = "...",  
    temperature = 0.7,  
    top_p = 0.9,  
    top_k = 50, # optional  
    min_p = 0.1, # optional  
    max_tokens = 256  
)
```

Creative writing

$T = 0.8\text{--}1.0$
 $\text{top-}p = 0.9\text{--}0.95$

Factual / Code

$T = 0.0\text{--}0.3$
or greedy ($T=0$)

Translation

Beam search, $B = 4\text{--}6$
length penalty $\alpha \approx 0.6$

Caveat: $T=0$ is *reproducible* only if the backend is fixed too. On a router (e.g. OpenRouter) different providers / quantization give different $T=0$ outputs — pin it: `provider:{order:[...], allow_fallbacks:false}`. Even pinned, batched GPU kernels are not bit-exact — expect *near-reproducibility*, not a hard guarantee.

Common questions

Q. Is $T = 0$ exactly greedy?

Yes — the softmax collapses onto the arg max. Libraries special-case $T=0$ so they don't divide by zero.

Q. If I set both top- k and top- p , which wins?

Both apply: you sample from their *intersection*, after temperature has reshaped the distribution.

Q. Is temperature applied before or after top- k /top- p ?

Before. Temperature rescales the logits; truncation then cuts the tail of the reshaped distribution.

Q. Is contrastive search a kind of sampling?

No — it is deterministic. It takes the arg max of a confidence-minus-penalty score, not a random draw.

Q. Does speculative decoding change the output?

No. It is lossless: identical to sampling the target model alone, only faster.

Further reading

Decoding methods

- ▶ Holtzman et al. (2020), “The Curious Case of Neural Text Degeneration” (nucleus / top- p)
- ▶ Fan et al. (2018), “Hierarchical Neural Story Generation” (top- k)
- ▶ Su et al. (2022), “A Contrastive Framework for Neural Text Generation” (contrastive search)

Analysis & newer methods

- ▶ Meister et al. (2020), “If Beam Search is the Answer, What was the Question?”
- ▶ Nguyen et al. (2024), “Turning up the Heat: Min- p Sampling”
- ▶ Leviathan et al. (2023), “Fast Inference from Transformers via Speculative Decoding”

Constrained decoding & practice

- ▶ Willard & Louf (2023), “Efficient Guided Generation for LLMs” (Outlines)
- ▶ HuggingFace blog, “How to generate text: using different decoding methods”
- ▶ Jurafsky & Martin, *Speech and Language Processing*, Ch. 10

Questions?

Next: Evaluation — Perplexity, BLEU, ROUGE